

Sprint 1 (Weeks 1-2): Foundation & Static UI

Tickets:

- Initialize React project
- Define the JSON Data Contract for communication with the Processor (Iris)
- Build the NewsCard UI and populate it using the 3-5 manual JSON sample articles provided by Iris
- Implement the main DashboardLayout to render the list of news items

Notes:

- Use Iris's manual samples instead of mock data to stay aligned from day one.
- Ensure the UI handles Hebrew titles and sources correctly.

Deliverable:

- A static React dashboard displaying actual sample data from the Processor's schema.

Sprint 2 (Weeks 3-4): Interactive Logic & Warning System

Tickets:

- **Edit Mode:** Build a text editor for Michal to refine the Hebrew title and publication text.
- **Dual-Text View:** Update NewsCard to show the "Short Summary" (2-3 sentences) in the feed and the "Publication Text" in the editor.
- **Warning UI:** Implement visual "Badges" for Processor-generated warnings (e.g., "Unclear Content", "Short Article") .
- **Action Logic:** Add buttons for decision making: "Not Relevant", "Website", "Newsletter", "Both" .

Notes:

- Focus on making the "Needs Review" status visually distinct

Deliverable: Interactive dashboard prototype with editing capabilities and warning indicators.

Sprint 3 (Weeks 5-6): Live Data Integration

Tickets:

- Integrate Axios/Fetch to pull real Hebrew outputs from the Processor's API/Module.
- Status Management: Map the Processor's statuses (processed, failed(?), needs review) to the UI
- Persistence: Implement the "Save" logic to update the shared database with Michal's decisions and edited texts.

- Handle API errors and loading states gracefully (e.g., if the Processor is slow or times out).

Notes:

- Account for asynchronous data loading and implement error handling for API failures.
- Ensure the DB architecture supports the statuses updated by the Dashboard.

Deliverable: A functional integrated system where the Dashboard pulls live data and pushes updates to the Backend.

Sprint 4 (Weeks 7-8): Security & Production Deployment

Tickets:

- Authentication: Add a secure login screen for Michal and the association staff.
- Publisher Hand-off: Finalize the data export format for Ran (the Publisher) based on Michal's selections.
- Batch Support: (Optional) Add a view for Michal to see the progress of items being processed in batches.
- Full E2E Test: Test the complete flow from news scraping (Ben) to publishing (Ran).
- Deployment: Deploy the frontend to Vercel/Netlify for production use.

Notes:

- Verify the final triggers for WordPress and Newsletter automation.
- Ensure all references and source links are properly tracked and displayed.

Deliverable: A production-ready, secured Dashboard deployed and accessible to the user.